

NovaCart: E-Commerce DBMS Architecture & EER Modeling Report

A Comprehensive Engineering & System Workflow Guide for the DBMS Innovative Examination (IE)

Course: Database Management Systems (CS301 / DBMS IE)	Engine: SQLite 3.x with WAL Mode
Focus: Conceptual EER Modeling, Schema Mapping & ACID	Integrity: PRAGMA foreign_keys = ON

1. Problem Statement & Conceptual Motivation

Commercial multi-vendor e-commerce platforms must support two fundamentally distinct user personas: **Buyers (Customers)** and **Merchants (Sellers)**. A naive database design that stores all individuals in a single flat table produces severe relational defects:

Flawed Flat-Table Approach	NovaCart EER Relational Solution
Sparse NULL Proliferation: Storing Customer columns (shipping address, phone, loyalty tier) and Seller columns (GSTIN tax number, store name, rating) in one table leaves 50% of attributes NULL for every row.	EER Disjoint Specialization: Base identity attributes live in superclass Users. Subclasses Customers and Sellers inherit user_id as both Primary Key and Foreign Key. Zero NULLs.
Redundancy & Data Inconsistency: Storing merchant branding across every order row duplicates seller information across hundreds of records. Deleting orders can accidentally purge buyer records.	Referential Integrity Rules: Managed via ON DELETE CASCADE for order/subclass cleanups and ON DELETE RESTRICT for catalog products referenced in order items.
Multi-Valued Attribute Proliferation: Storing comma-separated product IDs inside order rows prevents direct relational foreign key enforcement and breaks relational join operations.	M:N Bridge Decomposition: In ER modeling, the Many-to-Many 'contains' relationship is decomposed into associative entity Order_Items with relationship attributes (qty, price, subtotal).

2. Conceptual ER & EER Modeling Architecture

The conceptual architecture is constructed according to formal Chen and EER notation standards:

Construct	Entity / Rel	Cardinality & Participation	DBMS Architectural Purpose
Superclass	Users	1:1 with Subclasses Total Participation	Generalizes shared attributes (user_id, full_name, email, role, created_at).
Disjoint Node	Disjoint 'd'	Exclusive Membership	Specifies that a User tuple can belong to at most one subclass (Customer XOR Seller).

Subclass	Customers	1:N with Orders Partial Participation	Specialized buyer entity (shipping_address, phone_number, loyalty_tier). PK=FK=user_id.
Subclass	Sellers	1:N with Products Partial Participation	Specialized merchant entity (gst_number, shop_name, rating, category). PK=FK=user_id.
Strong Entity	Products	1:N with Sellers; M:N with Orders Total with Sellers	Store merchandise catalog. Enforces stock integrity with CHECK(stock_quantity >= 0).
Strong Entity	Orders	1:N with Customers; M:N with Products Total with Customers	Transactional checkout header storing timestamp, gross total, and delivery status.
Bridge Entity	Order_Items	Resolves M:N ('contains') Total with Orders & Products	Decomposes M:N into two 1:N relations with relationship attributes (qty, price, subtotal).

3. Physical Relational Schema & ER-to-Relational Mapping

The conceptual ER/EER model is mapped into a physical relational schema using standard **ER-to-Relational Mapping Rules**:

- **Rule 1 (Specialization / Inheritance Mapping):** The superclass Users and subclasses Customers and Sellers each map to separate physical tables. The subclass primary key is identical to the superclass primary key (user_id) and acts simultaneously as a Foreign Key with ON DELETE CASCADE, enforcing total disjoint (d) specialization with zero NULL columns.
- **Rule 2 (1:N Relationship Mapping):** For 1:N relationships (*places* between Customers & Orders; *offers* between Sellers & Products), the primary key of the '1' entity is placed as a Foreign Key in the 'N' entity (customer_id in Orders; seller_id in Products).
- **Rule 3 (M:N Relationship Decomposition):** The Many-to-Many *contains* relationship between Orders and Products is mapped to an associative (bridge) relation Order_Items. It includes Foreign Keys referencing both parent tables alongside relationship-specific attributes (quantity, unit_price, subtotal).
- **Rule 4 (Integrity & Constraint Enforcement):** Entity integrity (Primary Keys), referential integrity (Foreign Keys with CASCADE/RESTRICT policies), and domain constraints (CHECK(stock_quantity >= 0), CHECK(role IN (...))) are physically enforced at the database engine level.

Physical DDL Schema Definition (SQLite 3.x):

```
CREATE TABLE Users (  
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    full_name TEXT NOT NULL,  
    email TEXT UNIQUE NOT NULL,  
    role TEXT CHECK(role IN ('Customer', 'Seller')) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE Customers (  
    user_id INTEGER PRIMARY KEY,  
    shipping_address TEXT NOT NULL,  
    phone_number TEXT NOT NULL,  
    loyalty_tier TEXT CHECK(loyalty_tier IN ('Bronze', 'Silver', 'Gold', 'Platinum')) DEFAULT 'Bronze',  
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE  
);  
  
CREATE TABLE Sellers (  
    user_id INTEGER PRIMARY KEY,  
    gst_number TEXT UNIQUE NOT NULL,  
    shop_name TEXT NOT NULL,  
    rating REAL DEFAULT 4.5,  
    business_category TEXT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE  
);  
  
CREATE TABLE Products (  
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    seller_id INTEGER NOT NULL,  
    name TEXT NOT NULL, category TEXT NOT NULL, price REAL NOT NULL,  
    stock_quantity INTEGER NOT NULL CHECK(stock_quantity >= 0),  
    sku TEXT UNIQUE NOT NULL, description TEXT,  
    FOREIGN KEY (seller_id) REFERENCES Sellers(user_id) ON DELETE RESTRICT  
);  
  
CREATE TABLE Orders (  
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    customer_id INTEGER NOT NULL,  
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    total_amount REAL NOT NULL CHECK(total_amount >= 0),  
    status TEXT CHECK(status IN ('Placed', 'Processing', 'Delivered')) DEFAULT 'Placed',  
    FOREIGN KEY (customer_id) REFERENCES Customers(user_id) ON DELETE CASCADE  
);  
  
CREATE TABLE Order_Items (  
    item_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    order_id INTEGER NOT NULL, product_id INTEGER NOT NULL,  
    quantity INTEGER NOT NULL CHECK(quantity > 0),  
    unit_price REAL NOT NULL, subtotal REAL NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE CASCADE,  
    FOREIGN KEY (product_id) REFERENCES Products(product_id) ON DELETE RESTRICT  
);
```

4. Complete End-to-End System Workflow

The system lifecycle operates across five distinct phases, linking user input, database transactions, and multi-table relational calculus:

Phase 1: Database Dependency Bootstrapping

SQLite initializes the tables in dependency order: Users → {Customers, Sellers} → Products → Orders → Order_Items. Foreign keys and WAL logging (Write-Ahead Logging) are activated to enforce ACID properties.

Phase 2: EER Disjoint Inheritance Pipeline

When a user signs up, the system executes an atomic two-step insertion: Step 1 inserts into Users, returning user_id. Step 2 inserts into Customers or Sellers using that exact same user_id as both Primary Key and Foreign Key.

Phase 3: Merchant Catalog Lifecycle (1:N 'offers')

Sellers add products to the catalog. Products.seller_id references Sellers(user_id). Stock integrity is protected by CHECK(stock_quantity >= 0), and historical audits are guaranteed by ON DELETE RESTRICT.

Phase 4: Transactional Checkout & Decomposition (M:N 'contains')

When a buyer checks out, an atomic transaction executes: (1) BEGIN TRANSACTION; (2) Validate stock inventory; (3) INSERT INTO Orders; (4) INSERT INTO Order_Items to materialize the M:N bridge; (5) UPDATE Products to decrement stock; (6) COMMIT. If stock is insufficient, a ROLLBACK aborts changes with zero dirty rows.

Phase 5: Multi-Tier Relational Join Traversal

The query engine traverses across all six relations in a single execution: Users ■ Customers ■ Orders ■ Order_Items ■ Products ■ Sellers. This reconstructs complete transaction histories without data loss or sparse NULL values.

5. Step-by-Step Prototype Demonstration Guide

Step & Tab	Action to Perform in Prototype	DBMS Academic Concept to Explain
Step 1: Tab 1 Canvas	Inspect the vector Chen diagram. Select entities from the dropdown inspector.	Explain the Chen symbols: Rectangles (Entities), Diamonds (1:N, M:N), Circle 'd' (Disjointness), and Total vs Partial participation.
Step 2: Tab 2 Inheritance	Select an existing user to view their equi-join. Register a new Customer or Seller using the form.	Demonstrate that subclasses inherit the superclass key (user_id as PK/FK) and that multi-table insertion occurs atomically.
Step 3: Tab 3 Marketplace	Select a Customer and Product, enter quantity, and click 'Execute Transactional Checkout'.	Walk through the execution timeline: creation of Orders, materialization of Order_Items, and stock decrements under ACID isolation.

Step 4: Tab 4 Join Graph	Show the 6-tier joined table and the displayed SQL query below.	Prove that decomposed relational tables can be joined back cleanly across foreign keys without duplicate rows, loss of information, or sparse NULL columns.
Step 5: Tab 5 Schema	Inspect physical SQLite tables and raw CREATE TABLE DDL statements.	Highlight constraints: PRIMARY KEY, FOREIGN KEY, ON DELETE CASCADE, ON DELETE RESTRICT, and CHECK constraints.

6. Examiner Viva Defense & Oral Q&A; Suite

Q1: Why did you choose Disjoint Specialization ('d') over Overlapping Specialization ('o')?

Model Answer: In our domain, buyer and merchant legal responsibilities are bifurcated. Customers require physical delivery addresses and loyalty tiers, while sellers require tax registration (GSTIN) and shop branding. Disjoint specialization ensures zero sparse NULL columns while preserving 1:1 referential integrity via shared user_id.

Q2: How does a Subclass inherit the Primary Key of a Superclass in SQLite?

Model Answer: In Customers and Sellers, user_id acts as BOTH the Primary Key and a Foreign Key referencing Users(user_id) ON DELETE CASCADE. Because the subclass PK is also a foreign key, an entity cannot exist in a subclass without a corresponding superclass tuple.

Q3: How is the Many-to-Many (M:N) relationship between Orders and Products mapped to relational tables?

Model Answer: In relational modeling, a Many-to-Many relationship cannot be represented directly with a single foreign key without using repeating columns or multi-valued fields. Instead, standard ER-to-relational mapping decomposes the M:N relationship into an associative (bridge) entity: Order_Items. This establishes two clean 1:N relationships (Orders 1:N Order_Items and Products 1:N Order_Items) while storing relationship-specific attributes such as quantity, unit price, and line subtotal.

Q4: What is the difference between ON DELETE CASCADE and ON DELETE RESTRICT in your schema?

Model Answer: ON DELETE CASCADE is used on Orders and Subclasses: deleting a user automatically removes their associated customer/seller records, preventing orphan tuples. In contrast, ON DELETE RESTRICT is used on Products(seller_id): a seller cannot be deleted if active catalog items or historical order lines reference them.

Q5: How does the system guarantee database consistency during an order checkout?

Model Answer: Operations are wrapped in an atomic multi-statement transaction (BEGIN TRANSACTION ... COMMIT). SQLite WAL mode ensures transactional isolation, while CHECK(stock_quantity >= 0) guarantees stock cannot fall below zero. If inventory is insufficient, a ROLLBACK aborts changes completely.